Sequence Recognition & OCR

Week 15 — Session 2: CRNN, CTC, decoding, and edit distance

Practical Image Processing

Lecture 15.2 ▪ 60–70 min

Recap of Session 1

- Segment-then-classify multiplies errors: a 97% digit reader is only $\sim$83% at 6 digits.
- Cutting fails on touching digits and odometer roll-over.
- End-to-end fix: CNN $\rightarrow$ a sequence of **feature columns** $\rightarrow$ RNN $\rightarrow$ per-column scores.
- Left open: the *alignment problem* — T columns vs. L characters, with the collapse rule as a hint.

Today's job

Make the CRNN concrete, meet **CTC** that trains it without cut marks, decode its output, and evaluate honestly with edit distance — then ship the reader.

Roadmap — Session 2

The CRNN in detail

CTC: training without alignment

Decoding & evaluation

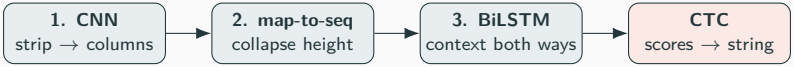
Code walkthrough

In-class exercise

Wrap-up

The CRNN in detail

The three stages of a CRNN



- **Convolutional** layers extract features while keeping the width axis.
- **Recurrent** layers read those columns left-to-right *and* right-to-left.
- The **transcription** layer (CTC) turns per-column scores into the final reading.

Stage 1: CNN, tracking the shape

Stage	**Output** $(C \times H \times W)$	**Note**
Input strip	$1 \times 8 \times 48$	6 digits, 8×8 each
Conv1 + pool 2×2	$16 \times 4 \times 24$	shrink both axes
Conv2 + pool 2×2	$32 \times 2 \times 12$	depth grows
Conv3 + pool 2×1	$32 \times 1 \times 12$	height $\rightarrow$ 1, width kept
Squeeze height	32×12	a length-12 sequence

- The final pool is 2×1: it removes the last of the height but leaves width alone.
- Result: T=12 columns, each a 32-d feature vector — the sequence the RNN reads.

Stage 2: map-to-sequence

- The CNN tensor is $C \times 1 \times T$. Squeeze the height, then **permute** to $T \times C$.
- Now the data is literally a sequence: "time" = horizontal position, "features" = channels.
- Column t carries what the CNN saw in a vertical slice around image-x $\approx 4t$ (its receptive field).

Math

tensor $(C, 1, T)$ $\rightarrow$ squeeze $\rightarrow$ (C, T) $\rightarrow$ permute $\rightarrow$ (T, C)

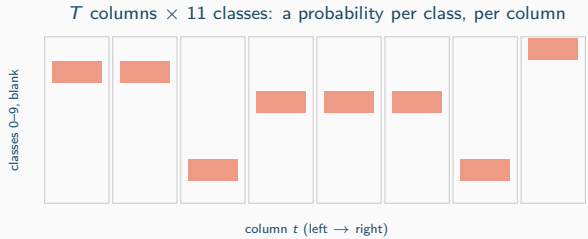
Stage 3: BiLSTM adds context

- Each column enters an **LSTM**, a recurrent unit that carries a memory along the sequence.
- **Bidirectional**: one pass left→right, one right→left; concatenate. Every column now knows what came before *and* after it.
- A linear layer maps each column's hidden state to **class scores**: 10 digits + 1 blank = 11.

✔ Good practice

This is why a half-rolled digit can still be read: its neighbours constrain what it must be.

The output we now have



- For every column we have a full probability distribution over the 11 classes.
- Coral cells = the most probable class per column (the "best path").
- But how do we *train* this, and turn it into one string? Enter CTC.

CTC: training without alignment

The blank symbol, precisely

- Add one extra class, the **blank** –: "emit no new character at this column."
- A raw per-column output (a **path**) is collapsed to text by two rules, *in order*:
  1. merge *adjacent equal* labels into one, then
  2. delete all blanks.
- The blank is what lets a genuine *double* letter survive.

Two paths, same length, different meaning

`7,7,7` $\to$ `7` but `7,-,7` $\to$ `77`

Many paths spell the same string

Target `42`, with $T{=}4$ columns and alphabet $\{4, 2, -\}$. All of these collapse to `42`:

path	collapses to	path	collapses to
`4,4,2,2`	`42`	`-,4,2,-`	`42`
`4,2,2,2`	`42`	`4,-,2,2`	`42`
`4,4,4,2`	`42`	`4,2,-,-`	`42`

- Many distinct column-paths correspond to the *one* reading we want.
- We never chose which path is "right" — and we don't have to.

The CTC idea: sum over all alignments

- Probability of a *path* = product of its per-column probabilities (columns treated as independent given the features).
- Probability of the *label* = sum of the probabilities of *all* paths that collapse to it.
- **CTC loss** = $-\log$ of that summed probability. Training pushes it up — for *every* valid alignment at once.

Connectionist Temporal Classification

$$P(\ell \mid x) = \sum_{\pi\,:\,\mathcal{B}(\pi)=\ell} \prod_{t=1}^{T} p_t(\pi_t), \qquad \text{loss} = -\log P(\ell \mid x)$$

Key idea

No cut marks needed: CTC lets the network discover the alignment by rewarding *all* of them.

CTC by hand, tiny example

Alphabet $\{a, -\}$, T=2, target `a`. Say per-column probabilities are:

	$p(a)$	$p(-)$
column 1	0.7	0.3
column 2	0.6	0.4

Paths collapsing to `a`: `a,a`, `a,-`, `-,a`.

Math

$$P(\texttt{a}) = \underbrace{0.7{\cdot}0.6}_{aa} + \underbrace{0.7{\cdot}0.4}_{a-} + \underbrace{0.3{\cdot}0.6}_{-a} = 0.42 + 0.28 + 0.18 = 0.88$$

Loss $= -\log 0.88 \approx 0.128$. The only excluded path is `-,-` (collapses to empty).

Why summing is tractable

- The number of paths is exponential in T — we cannot enumerate them for real strips.
- But the collapse rule has structure: a **dynamic-programming** pass (the "forward algorithm") sums all alignments in $O(T \cdot L)$ time.
- You never implement this yourself — `nn.CTCLoss` does it, differentiably.

✓ Good practice

Same trick as summing paths through a lattice: reuse partial sums instead of re-walking every route.

Decoding & evaluation

Greedy (best-path) decoding

- At *inference* we want one string. Simplest: take the `argmax` class in each column...
- ...then apply the collapse rule (merge repeats, drop blanks). This is **greedy decoding**.
- Fast and usually good, but it commits column-by-column and can miss the truly best *string*.

Greedy on our example

argmax per column: `4,4,-,2,2,-,6,6` → merge → `4,-,2,-,6` → drop blanks → `426`

Beam search decoding

- Greedy can be wrong: the best *path* is not always the best *string* (many paths vote for a string).
- **Beam search** keeps the top-k partial strings, accumulating the summed probability per string.
- More faithful to the CTC objective; costs more compute. A language model can be folded in here.

✔ Rule of thumb

Start with greedy; reach for beam search only if the greedy reading is not accurate enough.

Evaluating a reader: not just “right/wrong”

- **Sequence accuracy**: fraction of readings that are *exactly* correct. Strict — one wrong digit fails the whole string.
- **Character error rate (CER)**: how many single-character edits to fix the reading, per character.
- CER is the humane metric: it credits a reading that is off by one digit instead of scoring it zero.

Key idea

Report both: sequence accuracy for “did we nail it,” CER for “how close were we.”

Edit distance (Levenshtein)

- The **edit distance** between two strings = fewest single-character *insertions*, *deletions*, *substitutions* to turn one into the other.
- Computed by a small dynamic-programming table — the same lattice idea as CTC's forward pass.

The recurrence

$$D[i,j] = \min \begin{cases} D[i-1,j] + 1 & \text{(delete)} \\ D[i,j-1] + 1 & \text{(insert)} \\ D[i-1,j-1] + \mathbb{1}[a_i \neq b_j] & \text{(match/sub)} \end{cases}$$

Edit distance, worked

Reference `018426`, prediction `018326`. Walk the strings:

pos	1	2	3	4	5	6
ref	0	1	8	4	2	6
pred	0	1	8	3	2	6

- One substitution ($4 \to 3$): edit distance $= 1$.
- CER $= 1/6 \approx 0.167$; sequence accuracy for this reading $= 0$ (not exactly correct).
- A pipeline that never gets the whole string but is usually off by one still has a *low CER*.

Insertions and deletions matter too

- If the reader drops a digit (`01826` for `018426`) that is a **deletion**, distance 1.
- If it hallucinates one (`0184426`) that is an **insertion**, distance 1.
- Length changes are exactly why we need edit distance, not a position-by-position compare.

✕ Pitfall

Comparing character-by-character at fixed positions breaks the moment the reading is one digit too short or too long — everything after the slip looks wrong. Edit distance handles it correctly.

Code walkthrough

The CRNN in PyTorch

```
class CRNN(nn.Module):
    def __init__(self, n_classes=11):   # 10 digits + blank
        super().__init__()
        self.cnn = nn.Sequential(
            nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 32, 3, padding=1), nn.ReLU(),
            nn.MaxPool2d((2, 1)))                  # height->1, width kept
        self.rnn = nn.LSTM(32, 64, bidirectional=True, batch_first=False)
        self.fc  = nn.Linear(128, n_classes)       # 64*2 -> classes
    def forward(self, x):                          # x: (B,1,8,W)
        f = self.cnn(x)                            # (B,32,1,T)
        f = f.squeeze(2).permute(2, 0, 1)          # (T,B,32)
        y, _ = self.rnn(f)                         # (T,B,128)
        return self.fc(y).log_softmax(2)           # (T,B,n_classes)
```

The three CRNN stages, line for line: `cnn`, then map-to-sequence, then `rnn`+`fc`.

Training with CTCLoss

```
ctc = nn.CTCLoss(blank=10, zero_infinity=True)
for xb, targets, in_lens, tgt_lens in loader:
    opt.zero_grad()
    logp = net(xb)                    # (T, B, 11) log-probs
    loss = ctc(logp, targets, in_lens, tgt_lens)
    loss.backward(); opt.step()
```

- `logp`: log-probabilities, shape (time, batch, classes) — CTC's required layout.
- `in_lens`: how many columns each sample really has (T); `tgt_lens`: each label's length (L).
- `blank=10`: the blank is class index 10. One call sums over *all* alignments for you.

✕ Pitfall

Getting `blank` index or the (T,B,C) order wrong is the #1 CTC bug — loss goes to `inf` or `nan`.

Greedy decode + edit distance

```python
def greedy_decode(logp, blank=10):          # logp: (T, n_classes)
    path = logp.argmax(1).tolist()          # best class per column
    out, prev = [], None
    for c in path:
        if c != prev and c != blank:        # merge repeats, drop blank
            out.append(c)
        prev = c
    return out

def edit_distance(a, b):                    # Levenshtein, DP table
    D = list(range(len(b) + 1))
    for i, ca in enumerate(a, 1):
        prev, D[0] = D[0], i
        for j, cb in enumerate(b, 1):
            prev, D[j] = D[j], min(D[j] + 1, D[j-1] + 1,
                                   prev + (ca != cb))
    return D[-1]
```

`greedy_decode` is the collapse rule in six lines; `edit_distance` is the recurrence from earlier.

What the demo prints

- `crnn_ctc.py` builds synthetic meter strips (random 3–6 digit concatenations of the Week 13 digits).
- Trains the CRNN with CTC on CPU, then greedy-decodes the test strips.
- Reports **sequence accuracy** and **CER**, and prints a few readings beside their truth.

✓ Honest scope

A teaching-scale demo: small net, synthetic strips, a few minutes on CPU. It shows CER *falling* and the collapse rule working — not a production meter reader.

In-class exercise

In-class exercise (10 min)

Reason about CTC, decoding, and metrics

1. Collapse the path `-,5,5,-,5,-` with the CTC rule. How many 5s in the final string, and why?
2. Alphabet $\{a, -\}$, T=2, target `aa`. Which single path spells `aa`, and what is its probability if $p_t(a)$=0.5 each column?
3. Reference `7013`, prediction `713`. Give the edit distance and the CER.
4. Your CTC loss immediately becomes `inf`. Name two likely causes.

Debrief

- (1) `-,5,5,-,5,-`: merge the adjacent `5,5`→`5`, keep the blank-separated `5`: `5,5` ⇒ **two** 5s. The blank between them prevents the merge.
- (2) Only `a,a` spells `aa` with $T{=}2$ (a blank would shorten it); probability $0.5 \cdot 0.5 = 0.25$.
- (3) One deletion (0 dropped): distance 1; CER $= 1/4 = 0.25$.
- (4) Wrong `blank` index (or logits not log-softmaxed), or an input length $T < L$ so no valid alignment exists.

Key idea

You can now hand-collapse paths, compute a tiny CTC probability, and score a reading with edit distance — the whole reading loop.

Wrap-up

What you should now be able to do

- Draw the CRNN — CNN feature columns, map-to-sequence, BiLSTM, transcription — and track its shapes.
- Explain CTC as summing over all alignments, with the blank and collapse rule.
- Decode greedily (and know when beam search helps).
- Evaluate a reader with edit distance, CER, and sequence accuracy — and say why CER is fairer.

This week's deliverable

Due before the finale (see `project/`)

1. Run `crnn_ctc.py`: train the CRNN+CTC reader on synthetic meter strips; report sequence accuracy and CER.
2. Show 5 decoded readings beside their ground truth; identify one insertion/deletion/substitution error.
3. Compare end-to-end reading against a segment-then-classify baseline (reuse your Week 13 CNN).
4. Vary strip length (3 vs. 6 digits) and report how CER changes.
5. Reflect: how does this complete DocScan's rectify $\rightarrow$ detect $\rightarrow$ **read** pipeline, and what would real meter photos need?

CTC lets the network align itself.

CNN columns, an RNN for context, and a loss that sums over every alignment — DocScan reads the whole number.